

**Министерство науки и высшего образования Российской Федерации**  
Федеральное государственное автономное образовательное учреждение  
высшего образования

**«Пермский национальный исследовательский политехнический  
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и  
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

## **ОТЧЕТ**

### **Лабораторная работа №3**

**«Классы и объекты. Перегрузка операторов.»**

**По дисциплине «Основы алгоритмизации и программирования»**

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

## ПОСТАНОВКА ЗАДАЧИ

1. Определить пользовательский класс.
2. Определить в классе следующие конструкторы: без параметров, с параметрами, копирования.
3. Определить в классе деструктор.
4. Определить в классе компоненты-функции для просмотра и установки полей данных (селекторы и модификаторы).
5. Перегрузить операцию присваивания.
6. Перегрузить операции ввода и вывода объектов с помощью потоков.
7. Перегрузить операции указанные в варианте.
8. Написать программу, в которой продемонстрировать создание объектов и работу всех перегруженных операций.

Создать класс Pair (пара чисел). Пара должна быть представлено двумя полями: типа `int` для первого числа и типа `double` для второго. Первое число при выводе на экран должно быть отделено от второго числа двоеточием. Реализовать:

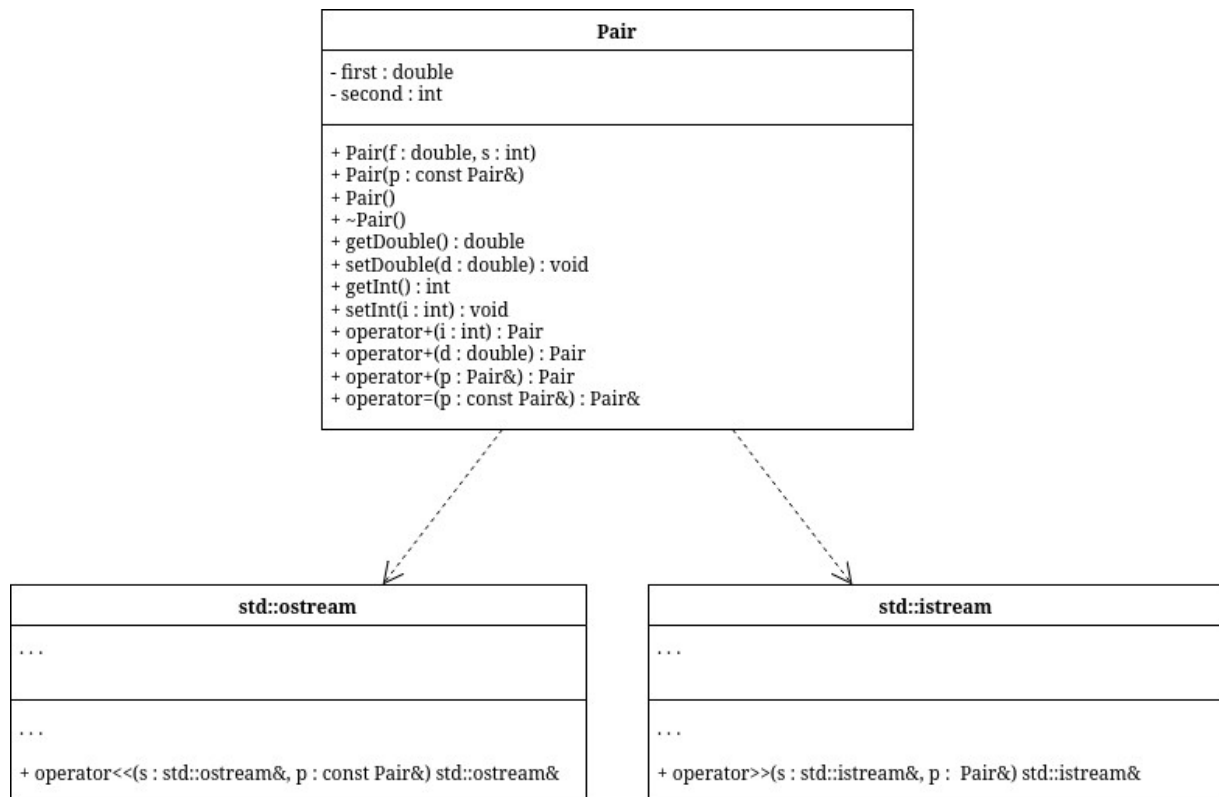
- а) вычитание пар чисел
- б) добавление константы к паре (увеличивается первое число, если константа целая, второе, если константа вещественная).

## АНАЛИЗ.

1. Согласно принципам инкапсуляции поля класса должны быть приватными, а описанные в постановке методы — публичными.
2. Для работы с приватными полями реализуем гетеры и сетеры.
3. Для объявления переменных заданного класса реализуем конструкторы: обычный (принимает на вход значения полей класса),

копирования и «пустой конструктор», выставляющий значения по умолчанию.

## UML-ДИАГРАММА



## КОД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#include <ostream>
```

```
class Pair {
    int first;
    double second;

public:
    Pair(double, int);
    Pair(Pair &);
    Pair();
    ~Pair();
    double getDouble() const;
    void setDouble(double);
    int getInt() const;
```

```

void setInt(int);
Pair operator+(Pair &p);
Pair operator+(int);
Pair operator+(double);
Pair &operator=(Pair const &p);
friend std::ostream &operator<<(std::ostream &, const Pair &);
friend std::istream &operator>>(std::istream &, Pair &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H

```

## КОД Pair.cpp

```

#include "Pair.h"
#include <iostream>

Pair::Pair(double a, int b) {
    first = b;
    second = a;
}

Pair::Pair(Pair &f) {
    second = f.second;
    first = f.first;
}

Pair::Pair() {
    second = 0;
    first = 0;
}

Pair::~~Pair() {
    second = 0;
    first = 0;
}

double Pair::getDouble() const {
    return second;
}

void Pair::setDouble(double a) {
    second = a;
}

```

```

int Pair::getInt() const {
    return first;
}

void Pair::setInt(int a) {
    first = a;
}

Pair &Pair::operator=(Pair const &p) {
    second = p.second;
    first = p.first;
    return *this;
}

std::ostream &operator<<(std::ostream &stream, const Pair &p) {
    std::cout << p.first << " : " << p.second << '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, Pair &p) {
    std::cout << "Double? ";
    stream >> p.second;
    std::cout << "Int? ";
    stream >> p.first;
    return stream;
}

Pair Pair::operator+(Pair &p) {
    Pair second(*this);
    second.second += p.second;
    second.first += p.first;
    return second;
}

Pair Pair::operator+(int a) {
    Pair tmp(*this);
    tmp.first += a;
    return tmp;
}

Pair Pair::operator+(double a) {
    Pair tmp(*this);
    tmp.second += a;
    return tmp;
}

```

```
}
```

## КОД main.cpp

```
#include <iostream>
#include "Pair.h"

using namespace std;

void addConst(Pair &p, int a) {
}

int main() {
    Pair pair1;
    cout << "Pair 1?\n";
    cin >> pair1;
    cout << ++pair1;
    cout << pair1;
    Pair pair2;
    cout << "Pair 2?\n";
    cin >> pair2;
    cout << pair2;
    cout << "Sum: " << pair1 + pair2 << "\n";
    int i;
    cout << "Integer? ";
    cin >> i;
    cout << "Pair 1 + integer: " << pair1 + i;
    double d;
    cout << "Double? ";
    cin >> d;
    cout << "Pair 2 + double: " << pair2 + d;
}
```

## РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Pair 1?

Double? 3.14

Int? 15

15 : 3.14

Pair 2?

Double? 2.34

Int? 8

8 : 2.34

Sum: 23 : 5.48

Integer? 10

Pair 1 + integer: 25 : 3.14

Double? 0.22

Pair 2 + double: 8 : 2.56

## ВОПРОСЫ

1. Для чего используются дружественные функции и классы?

Для доступа к приватным полям вне методов класса.

2. Сформулировать правила описания и особенности дружественных функций.

- Объявляется внутри класса, не имеет доступа к `this`
- Может быть любой функцией или любым методом другого класса
- Может быть дружественна к нескольким классам
- Не распространяется спецификатор доступа

3. Каким образом можно перегрузить унарные операции?

- Как компонентную функцию класса
- Как глобальную функцию

4. Сколько операндов должна иметь унарная функция-операция, определяемая внутри класса?

Ни сколько, она имеет доступ к `this`

5. Сколько операндов должна иметь унарная функция-операция, определяемая вне класса?

Один, так как не имеет доступа к `this`

6. Сколько операндов должна иметь бинарная функция-операция, определяемая внутри класса?

Один, имеет доступ к `this`

7. Сколько операндов должна иметь бинарная функция-операция, определяемая вне класса?

Два, не имеет доступа к `this`

8. Чем отличается перегрузка префиксных и постфиксных унарных операций?

Наличием фиктивного параметра `int` и префиксной

9. Каким образом можно перегрузить операцию присваивания?

```
Pair &Pair::operator=(Pair const &p) {  
    second = p.second;  
    first = p.first;  
    return *this;  
}
```

10. Что должна возвращать операция присваивания?

Ссылку на класс, в котором объявлена.

11. Каким образом можно перегрузить операции ввода-вывода?



Создав friend перегрузку операторов << и >> для ostream и istream соответственно.

12. В программе описан класс

```
class Student
```

```
{
```

```
...
```

```
Student& operator++();
```

```
....
```

```
};
```

и определен объект этого класса

```
Student s;
```

Выполняется операция

```
++s;
```

Каким образом, компилятор будет воспринимать вызов функции-операции?

Префиксный инкремент, метод

13. В программе описан класс

```
class Student
```

```
{
```

```
...
```

```
friend Student& operator ++( Student&);
```

```
....
```

```
};
```

и определен объект этого класса

```
Student s;
```

Выполняется операция

```
++s;
```

Каким образом, компилятор будет воспринимать вызов функции-операции?

Префиксный инкремент, дружественная функция

14. В программе описан класс

```
class Student
```

```
{
```

```
...
```

```
bool operator<(Student &P);
```

```
....
```

```
};
```

и определены объекты этого класса

```
Student a,b;
```

Выполняется операция

```
cout<<a<b;
```

Каким образом, компилятор будет воспринимать вызов функции-операции?

Как операцию сравнения, вернёт true/false

15. В программе описан класс

```
class Student
{
...
friend bool operator >(const Person&, Person&)
....
};
```

и определены объекты этого класса

```
Student a,b;
```

Выполняется операция

```
cout<<a>b;
```

Каким образом, компилятор будет воспринимать вызов функции-операции?

Как операцию сравнения, вернёт true/false